PHP OOP

Code Examples & Exercises (Topics 1-26)

Requires PHP 8.1+. Run each file with: `php filename.php`

01	Class	`01-class.php`
02	Constants Properties Methods	`02-constants-properties-methods.php`
03	Constructor	`03-constructor.php`
04	Destructor	`04-destructor.php`
05	Access Modifiers	`05-access-modifiers.php`
06	Scope Resolution	`06-scope-resolution.php`
07	Object Instances	`07-object-instances.php`
08	Member Visibility	`08-member-visibility.php`
09	Static	`09-static.php`
10	Inheritance	`10-inheritance.php`
11	Interfaces	`11-interfaces.php`
12	Abstraction	`12-abstraction.php`
13	Overrides	`13-overrides.php`
14	Final	`14-final.php`
15	Type Hinting	`15-type-hinting.php`
16	Magic Methods	`16-magic-methods.php`
17	Objects References	`17-objects-references.php`
18	Cloning	`18-cloning.php`
19	Comparing Objects	`19-comparing-objects.php`
20	Late Static Binding	`20-late-static-binding.php`
21	Traits	`21-traits.php`

22	Namespace	`22-namespace.php`
23	Builtin Interfaces	`23-builtin-interfaces.php`
24	Class Functions	`24-class-functions.php`
25	Serialization	`25-serialization.php`
26	Exceptions	`26-exceptions.php`

01. Class

01-class.php

```php
<?php
// Class = blueprint (naksha), Object = oi naksha theke banano jinis.
class Car {
    // property (variable) ar method (function) class er vitore thake
    public string $brand = 'Toyota';

    public function drive(): string {
        return "{$this->brand} is driving";   // $this = current object
    }
}

$car = new Car();            // class theke object toiri
echo $car->drive() . PHP_EOL;
echo $car->brand . PHP_EOL;

// Class er nam convention: PascalCase (Car, UserProfile). File e ek class thakle bhalo.
// EXERCISE: Student class banan (name property, introduce() method).
```

02. Constants Properties Methods

02-constants-properties-methods.php

```
<?php
class Circle {
    const PI = 3.14159;                 // class constant (ager $ lage na)
    public float $radius = 1.0;         // property
    public static int $count = 0;       // static property

    public function area(): float {     // method
        return self::PI * $this->radius ** 2;
    }
    public function setRadius(float $r): void {
        $this->radius = $r;
    }
}

$c = new Circle();
$c->setRadius(5);
echo $c->area() . PHP_EOL;
echo Circle::PI . PHP_EOL;              // constant e object lage na
echo $c->radius . PHP_EOL;

// Typed + readonly property (PHP 8.1)
class Book {
    public function __construct(public readonly string $title) {}
}
$b = new Book('PHP OOP');
// $b->title = 'x';   // Error: readonly

// EXERCISE: Rectangle class banan (width, height property, area() ar perimeter() method).
```

03. Constructor

03-constructor.php

```
<?php
class User {
    public string $name;
    public int $age;

    // object toiri hoyar shathe shathe auto cholbe
    public function __construct(string $name, int $age = 18) {
        $this->name = $name;
        $this->age  = $age;
        echo "User created: $name\n";
    }
}
$u = new User('Rahim', 25);
$u2 = new User('Karim');       // age default 18
echo $u2->age . PHP_EOL;

// Constructor property promotion (PHP 8): chhoto kore likha
class Product {
    public function __construct(
        public string $title,
        public float $price = 0.0,
        protected int $stock = 0,
    ) {}
}
$p = new Product('Pen', 10.5);
echo $p->title . ' - ' . $p->price . PHP_EOL;

// Named arguments (PHP 8)
$p2 = new Product(title: 'Book', price: 250);

// EXERCISE: Employee class banan promotion diye (name, salary, department).
```

04. Destructor

04-destructor.php

```
<?php
class FileHandler {
    private $handle;

    public function __construct(private string $path) {
        $this->handle = fopen($path, 'w');
        echo "File opened\n";
    }
    public function write(string $text): void { fwrite($this->handle, $text); }

    // object memory theke jawar age auto cholbe (cleanup er jonno)
    public function __destruct() {
        fclose($this->handle);
        echo "File closed\n";
    }
}

$f = new FileHandler(sys_get_temp_dir() . '/demo.txt');
$f->write('Hello');

$f = null;                      // reference remove -> destructor ekhoni cholbe
echo "After null\n";

$g = new FileHandler(sys_get_temp_dir() . '/demo2.txt');
echo "Script ending...\n";
// script shesh hole $g er destructor auto cholbe

// Use: DB connection, file handle, lock release.
// EXERCISE: Logger class banan jeta destruct e "Logger closed" print kore.
```

05. Access Modifiers

05-access-modifiers.php

```php
<?php
class BankAccount {
    public string $owner;            // jekhane khushi access
    protected float $balance = 0;    // nijer class + child class
    private string $pin = '1234';    // shudhu nijer class

    public function __construct(string $owner) { $this->owner = $owner; }

    public function deposit(float $amt): void { $this->balance += $amt; }
    public function getBalance(): float { return $this->balance; }
    private function checkPin(string $p): bool { return $p === $this->pin; }
    public function verify(string $p): bool { return $this->checkPin($p); }
}

$acc = new BankAccount('Asif');
$acc->deposit(1000);
echo $acc->owner . PHP_EOL;          // public: OK
echo $acc->getBalance() . PHP_EOL;   // getter diye access
var_dump($acc->verify('1234'));      // private method public method er vitor diye

try { echo $acc->balance; }          // protected: bahire error
catch (Error $e) { echo "Error: " . $e->getMessage() . PHP_EOL; }

try { echo $acc->pin; }              // private: bahire error
catch (Error $e) { echo "Error: " . $e->getMessage() . PHP_EOL; }

// Table:
// public     -> everywhere
// protected  -> same class + child class
// private    -> same class only

// EXERCISE: Student class e private $marks rakhun, setMarks() e 0-100 validation dien.
```

06. Scope Resolution

06-scope-resolution.php

```
<?php
class Config {
    const APP = 'MyApp';
    public static string $env = 'dev';

    public static function info(): string {
        return self::APP . ' in ' . self::$env;     // self:: = ei class
    }
}
echo Config::APP . PHP_EOL;          // constant
echo Config::$env . PHP_EOL;         // static property ($ lage)
echo Config::info() . PHP_EOL;       // static method
echo Config::class . PHP_EOL;        // class er full nam

class Base {
    public function hello(): string { return 'Hello from Base'; }
    public static function create(): static { return new static(); }
}
class Child extends Base {
    public function hello(): string {
        return parent::hello() . ' + Child';        // parent:: = parent class
    }
}
echo (new Child())->hello() . PHP_EOL;
echo get_class(Child::create()) . PHP_EOL;          // static:: (details: Late Static Binding)

// self:: = je class e likha, parent:: = parent class, static:: = je class call korse
// EXERCISE: MathHelper class e const PI + static square($n) banan, :: diye call korun.
```

07. Object Instances

07-object-instances.php

```php
<?php
class Counter {
    public int $value = 0;
    public function inc(): static { $this->value++; return $this; }   // method chaining
}

$a = new Counter();
$b = new Counter();          // alada instance, alada data
$a->inc()->inc()->inc();     // chaining
$b->inc();
echo $a->value . ' ' . $b->value . PHP_EOL;     // 3 1

var_dump($a instanceof Counter);                 // true
echo get_class($a) . PHP_EOL;

// Dynamic class name
$className = 'Counter';
$c = new $className();

// Anonymous class
$anon = new class { public function hi() { return 'Hi from anon'; } };
echo $anon->hi() . PHP_EOL;

// Object theke array/stdClass
$obj = new stdClass();
$obj->name = 'Asif';
$obj->city = 'Dhaka';
print_r((array) $obj);

// EXERCISE: 3ta Product object banan, array te rekhe foreach e price print korun.
```

08. Member Visibility

08-member-visibility.php

```
<?php
class ParentClass {
    public $pub = 'public';
    protected $pro = 'protected';
    private $pri = 'private';

    public const A = 'pub const';
    protected const B = 'protected const';       // constant-o visibility pay (PHP 7.1+)
    private const C = 'private const';

    public function showAll(): string {
        return "$this->pub | $this->pro | $this->pri | " . self::A . ' | ' . self::B . ' | ' . self::C;
    }
    // same class er onno object er private-o access kora jay
    public function samePri(ParentClass $other): string { return $other->pri; }
}

class ChildClass extends ParentClass {
    public function childAccess(): string {
        return $this->pub . ' | ' . $this->pro . ' | ' . self::B;
        // $this->pri   -> child e private paoa jabe na
    }
}

$p = new ParentClass();
$c = new ChildClass();
echo $p->showAll() . PHP_EOL;
echo $c->childAccess() . PHP_EOL;
echo $p->samePri(new ParentClass()) . PHP_EOL;

echo $p->pub . PHP_EOL;                            // OK
try { echo ParentClass::B; } catch (Error $e) { echo "Error: " . $e->getMessage() . PHP_EOL; }

// Method visibility-o same rule. Child override e visibility kombaano jay na (public -> private Fatal).
// EXERCISE: ParentClass e protected method add korun, child theke call korun, bahire call kore error
    dekhun.
```

09. Static

09-static.php

```php
<?php
class Visitor {
    public static int $total = 0;          // class er shob object er majhe shared
    public string $name;

    public function __construct(string $name) {
        $this->name = $name;
        self::$total++;                    // static e self:: / ClassName::
    }
    public static function getTotal(): int {
        return self::$total;               // static method e $this nai
    }
}

new Visitor('A');
new Visitor('B');
new Visitor('C');
echo Visitor::getTotal() . PHP_EOL;        // 3
echo Visitor::$total . PHP_EOL;            // 3

// Utility/helper class
class Str {
    public static function slug(string $s): string {
        return strtolower(trim(preg_replace('/[^A-Za-z0-9]+/', '-', $s), '-'));
    }
}
echo Str::slug('Hello World PHP!') . PHP_EOL;   // hello-world-php

// Singleton pattern (static er common use)
class DB {
    private static ?DB $instance = null;
    private function __construct() { echo "DB connected\n"; }
    public static function get(): DB {
        return self::$instance ??= new DB();
    }
}
DB::get();
DB::get();     // "DB connected" ekbar-i print hoy
var_dump(DB::get() === DB::get());   // true

// EXERCISE: IdGenerator class banan, static next() jeta 1,2,3... return kore.
```

10. Inheritance

10-inheritance.php

```
<?php
class Vehicle {
    public function __construct(protected string $name, protected int $wheels = 4) {}

    public function describe(): string {
        return "{$this->name} has {$this->wheels} wheels";
    }
    public function start(): string { return "{$this->name} starts"; }
}

class Bike extends Vehicle {                        // extends = inherit
    public function __construct(string $name) {
        parent::__construct($name, 2);              // parent constructor manually call
    }
    public function wheelie(): string { return "{$this->name} does a wheelie!"; }   // new method
}

class ElectricCar extends Vehicle {
    public function __construct(string $name, private int $battery = 100) {
        parent::__construct($name);
    }
    public function start(): string {               // override
        return parent::start() . " silently (battery {$this->battery}%)";
    }
}

$b = new Bike('Yamaha');
$e = new ElectricCar('Tesla');
echo $b->describe() . PHP_EOL;     // parent theke pawa method
echo $b->wheelie() . PHP_EOL;
echo $e->start() . PHP_EOL;

var_dump($e instanceof Vehicle);   // true
print_r(class_parents($e));

// PHP e single inheritance (ekta-i parent). Multiple er jonno interface/trait.
// Multilevel: A -> B -> C hote pare.
// EXERCISE: Person -> Teacher -> HeadTeacher 3 level hierarchy banan.
```

11. Interfaces

11-interfaces.php

```
<?php
// Interface = contract. Class-ke ei method gulo implement korte-i hobe.
interface Payable {
    const CURRENCY = 'BDT';              // interface constant
    public function pay(float $amount): string;
}
interface Refundable {
    public function refund(float $amount): string;
}

// ekta class multiple interface implement korte pare
class Bkash implements Payable, Refundable {
    public function pay(float $amount): string {
        return "bKash: $amount " . self::CURRENCY . " paid";
    }
    public function refund(float $amount): string {
        return "bKash: $amount " . self::CURRENCY . " refunded";
    }
}
class Cash implements Payable {
    public function pay(float $amount): string {
        return "Cash: $amount " . self::CURRENCY . " received";
    }
}

// Payable type hint -> je kono Payable object cholbe (polymorphism)
function checkout(Payable $method, float $amount): void {
    echo $method->pay($amount) . PHP_EOL;
}
checkout(new Bkash(), 500);
checkout(new Cash(), 300);

var_dump(new Bkash() instanceof Refundable); // true
var_dump(new Cash() instanceof Refundable);  // false

// EXERCISE: Nagad class banan jeta Payable + Refundable duto-i implement kore.
```

12. Abstraction

12-abstraction.php

```
<?php
// abstract class: nijer object banano jay na, child ke abstract method implement korte hoy
abstract class Shape {
    public function __construct(protected string $name) {}

    abstract public function area(): float;   // body nai

    // concrete method: shob child use korbe
    public function describe(): string {
        return sprintf("%s area = %.2f", $this->name, $this->area());
    }
}
class Circle extends Shape {
    public function __construct(private float $r) { parent::__construct('Circle'); }
    public function area(): float { return M_PI * $this->r ** 2; }
}
class Rectangle extends Shape {
    public function __construct(private float $w, private float $h) { parent::__construct('Rectangle');
        }
    public function area(): float { return $this->w * $this->h; }
}

foreach ([new Circle(5), new Rectangle(4, 6)] as $s) {
    echo $s->describe() . PHP_EOL;
}

try {
    $x = new Shape('x');   // Error!
} catch (Error $e) {
    echo "Error: " . $e->getMessage() . PHP_EOL;
}

// abstract class vs interface:
//  - abstract class e property + concrete method thakte pare, single inheritance
//  - interface e shudhu contract, multiple implement kora jay
// EXERCISE: Triangle class banan (area = 0.5 * base * height).
```

13. Overrides

13-overrides.php

```
<?php
class Animal {
    public function __construct(protected string $name) {}
    public function speak(): string { return "{$this->name} makes a sound"; }
    public function info(): string  { return "Animal: {$this->name}"; }
}
class Dog extends Animal {
    // override: same name, compatible signature
    public function speak(): string { return "{$this->name} says Woof!"; }

    // parent:: diye parent er version-o call kora jay
    public function info(): string { return parent::info() . " (Dog)"; }
}

$d = new Dog('Tommy');
echo $d->speak() . PHP_EOL;  // Tommy says Woof!
echo $d->info()  . PHP_EOL;  // Animal: Tommy (Dog)

// Signature rule: parameter type widen kora jay, return type narrow kora jay
class A { public function get(int $x): iterable { return [$x]; } }
class B extends A { public function get(int|string $x): array { return [$x]; } } // OK

// EXERCISE: Cat class banan, speak() override korun, info() e parent:: use korun.
```

14. Final

14-final.php

```
<?php
class Config {
    final public const VERSION = '1.0';          // final const (PHP 8.1+)
    final public function appName(): string {     // override kora jabe na
        return 'MyApp';
    }
    public function debug(): bool { return false; }
}
class DevConfig extends Config {
    public function debug(): bool { return true; }   // OK
    // public function appName() {}                  // Fatal error: final method
}
echo (new DevConfig())->appName() . PHP_EOL;

final class Money {                // final class: extend kora jabe na
    public function __construct(public readonly int $amount) {}
}
// class Taka extends Money {}     // Fatal error

// Kokhon final? Security-sensitive ba immutable class (Money, Token) jekhane inheritance e behaviour
    bhenge jete pare.
// EXERCISE: ekta final class PasswordHasher banan, tarpor extend kore dekhun ki error ashe.
```

15. Type Hinting

15-type-hinting.php

```
<?php
declare(strict_types=1);   // strict mode: "5" ke int hishabe accept korbe na

function add(int $a, int $b): int { return $a + $b; }
function greet(?string $name = null): string {     // nullable
    return "Hello, " . ($name ?? 'Guest');
}
function show(int|string $id): string {            // union type (PHP 8)
    return "ID: $id";
}
function logMsg(string $m): void { echo $m . PHP_EOL; }   // void return

class User { public function __construct(public string $name) {} }
function welcome(User $u): string { return "Welcome {$u->name}"; }  // class type

echo add(2, 3) . PHP_EOL;
echo greet() . PHP_EOL;
echo show(10) . ' | ' . show('abc') . PHP_EOL;
echo welcome(new User('Asif')) . PHP_EOL;

try {
    echo add("5", 3);          // strict_types e TypeError
} catch (TypeError $e) {
    echo "TypeError: " . $e->getMessage() . PHP_EOL;
}

// Typed property
class Product {
    public int $qty = 0;
    public ?float $price = null;
}
// EXERCISE: strict_types off kore "5" pathan, ki hoy dekhun. Tarpor on kore abar.
```

16. Magic Methods

16-magic-methods.php

```
<?php
class Magic {
    private array $data = [];

    public function __construct()  { echo "constructed\n"; }
    public function __destruct()   { echo "destroyed\n"; }

    public function __set($k, $v)  { $this->data[$k] = $v; }
    public function __get($k)      { return $this->data[$k] ?? "(no $k)"; }
    public function __isset($k)    { return isset($this->data[$k]); }
    public function __unset($k)    { unset($this->data[$k]); }

    public function __call($name, $args)        { return "call: $name(" . implode(',', $args) . ")"; }
    public static function __callStatic($n, $a) { return "static call: $n"; }

    public function __toString(): string { return "Magic(" . json_encode($this->data) . ")"; }
    public function __invoke($x)         { return $x * 2; }
}

$m = new Magic();
$m->name = 'Asif';              // __set
echo $m->name . PHP_EOL;        // __get
var_dump(isset($m->name));      // __isset
unset($m->name);                // __unset
echo $m->missing . PHP_EOL;     // (no missing)
echo $m->doSomething(1, 2) . PHP_EOL;     // __call
echo Magic::build() . PHP_EOL;            // __callStatic
echo $m . PHP_EOL;                        // __toString
echo $m(21) . PHP_EOL;                    // __invoke => 42
// script shesh e __destruct auto cholbe

// EXERCISE: __get diye ekta read-only Settings class banan jeta array theke value dey.
```

17. Objects References

17-objects-references.php

```
<?php
class Counter { public int $n = 0; }

// Object variable e "handle" thake, pura object na.
$a = new Counter();
$b = $a;           // same object er handle copy hoy
$b->n = 10;
echo $a->n . PHP_EOL;   // 10 (same object)
var_dump(spl_object_id($a) === spl_object_id($b)); // true

// Function e pass korle-o object modify hoy
function inc(Counter $c): void { $c->n++; }
inc($a);
echo $a->n . PHP_EOL;   // 11

// Kintu handle replace korle bahire effect porbe na
function reset1(Counter $c): void { $c = new Counter(); }
reset1($a);
echo $a->n . PHP_EOL;   // 11 (change hoy nai)

// & diye reference pass korle handle-o replace hoy
function reset2(Counter &$c): void { $c = new Counter(); }
reset2($a);
echo $a->n . PHP_EOL;   // 0

// EXERCISE: array of objects foreach e modify korun, tarpor foreach (&$x) diye compare korun.
```

18. Cloning

18-cloning.php

```php
<?php
class Address { public function __construct(public string $city) {} }
class Person {
    public function __construct(public string $name, public Address $address) {}

    // Shallow clone e nested object shared thake. Deep copy er jonno __clone
    public function __clone() {
        $this->address = clone $this->address;
    }
}

$p1 = new Person('Rahim', new Address('Dhaka'));
$p2 = clone $p1;           // __clone chole
$p2->name = 'Karim';
$p2->address->city = 'Sylhet';

echo $p1->name . ' - ' . $p1->address->city . PHP_EOL;   // Rahim - Dhaka
echo $p2->name . ' - ' . $p2->address->city . PHP_EOL;   // Karim - Sylhet

// __clone comment kore dekhun: p1 er city-o Sylhet hoye jabe (shallow copy)

// EXERCISE: Order class (items array of objects) banan, deep clone korun.
```

19. Comparing Objects

19-comparing-objects.php

```
<?php
class Point {
    public function __construct(public int $x, public int $y) {}
}
$a = new Point(1, 2);
$b = new Point(1, 2);
$c = $a;

var_dump($a == $b);    // true  : same class + same property value
var_dump($a === $b);   // false : alada object
var_dump($a === $c);   // true  : same instance

$b->y = 5;
var_dump($a == $b);    // false

// instanceof
class Admin extends Point {}
var_dump(new Admin(0, 0) instanceof Point);    // true
var_dump($a instanceof Admin);                 // false

// Custom equality: nijer method banano bhalo
class Money {
    public function __construct(public int $amount, public string $cur) {}
    public function equals(Money $o): bool {
        return $this->amount === $o->amount && $this->cur === $o->cur;
    }
}
var_dump((new Money(100, 'BDT'))->equals(new Money(100, 'BDT'))); // true

// EXERCISE: duita alada class er object same property niye == diye compare korun.
```

20. Late Static Binding

20-late-static-binding.php

```
<?php
class Model {
    public static function create(): static { return new static(); }   // static = je class call korse
    public static function createSelf(): self { return new self(); }   // self = je class e likha

    public static function who(): string { return self::class; }
    public static function lateWho(): string { return static::class; }

    public static function table(): string {
        return strtolower(static::class) . 's';
    }
}
class User extends Model {}
class Post extends Model {}

echo get_class(User::create()) . PHP_EOL;       // User
echo get_class(User::createSelf()) . PHP_EOL;   // Model (self)
echo User::who() . ' / ' . User::lateWho() . PHP_EOL;   // Model / User
echo User::table() . ' | ' . Post::table() . PHP_EOL;   // users | posts

// Laravel Eloquent e Model::find() ei concept e kaj kore.
// EXERCISE: Model e static function all() banan jeta static::table() use kore.
```

21. Traits

21-traits.php

```
<?php
trait Logger {
    public function log(string $m): void { echo "[LOG] $m\n"; }
}
trait Timestamp {
    public ?string $createdAt = null;
    public function touch(): void { $this->createdAt = date('Y-m-d H:i:s'); }
    public function log(string $m): void { echo "[TIME] $m\n"; }   // Logger er sathe conflict
}
trait Counts {
    private static int $count = 0;                    // static in trait
    public static function inc(): int { return ++self::$count; }
    abstract public function id(): int;               // abstract in trait
}

class Article {
    use Logger, Timestamp {
        Logger::log insteadof Timestamp;     // conflict resolve
        Timestamp::log as timeLog;           // alias
    }
    use Counts;
    public function id(): int { return 1; }
}

$a = new Article();
$a->log('saved');          // [LOG] saved
$a->timeLog('saved');      // [TIME] saved
$a->touch();
echo $a->createdAt . PHP_EOL;
echo Article::inc() . Article::inc() . PHP_EOL;   // 12

print_r(class_uses($a));

// Trait = code reuse (multiple inheritance er moto), interface = contract.
// EXERCISE: SoftDeletes trait banan (deleted_at, delete(), restore()).
```

22. Namespace

22-namespace.php

```
<?php
// Ek file e multiple namespace dekhanor jonno bracket syntax
namespace App\Models {
    class User { public function name(): string { return 'Model User'; } }
    function helper(): string { return 'models helper'; }
    const VERSION = '1.0';
}

namespace App\Services {
    class User { public function name(): string { return 'Service User'; } }
}

namespace {   // global namespace
    use App\Models\User;
    use App\Services\User as ServiceUser;      // alias
    use function App\Models\helper;
    use const App\Models\VERSION;

    echo (new User())->name() . PHP_EOL;
    echo (new ServiceUser())->name() . PHP_EOL;
    echo helper() . ' ' . VERSION . PHP_EOL;
    echo \App\Models\User::class . PHP_EOL;    // fully qualified name
    echo strlen('abc') . PHP_EOL;              // global function, \ lage na (fallback)
}

// Real project e: ek file = ek class, folder structure = namespace (PSR-4)
// composer.json:
//   "autoload": { "psr-4": { "App\\": "src/" } }
// src/Models/User.php  ->  namespace App\Models; class User {}
// Tarpor: composer dump-autoload  +  require 'vendor/autoload.php';

// EXERCISE: alada folder e 2ta class banan, PSR-4 autoload diye load korun.
```

23. Builtin Interfaces

23-builtin-interfaces.php

```
<?php
class Collection implements ArrayAccess, Countable, IteratorAggregate, JsonSerializable, Stringable {
    public function __construct(private array $items = []) {}

    // ArrayAccess: $c['a']
    public function offsetExists(mixed $o): bool { return isset($this->items[$o]); }
    public function offsetGet(mixed $o): mixed   { return $this->items[$o] ?? null; }
    public function offsetSet(mixed $o, mixed $v): void {
        if ($o === null) $this->items[] = $v; else $this->items[$o] = $v;
    }
    public function offsetUnset(mixed $o): void  { unset($this->items[$o]); }

    public function count(): int { return count($this->items); }                        // count($c)
    public function getIterator(): Iterator { return new ArrayIterator($this->items); } // foreach
    public function jsonSerialize(): mixed { return $this->items; }                     // json_encode
    public function __toString(): string { return implode(', ', $this->items); }       // Stringable
}

$c = new Collection(['a' => 'Apple']);
$c['b'] = 'Banana';
$c[] = 'Cherry';
echo $c['a'] . PHP_EOL;
echo count($c) . PHP_EOL;
foreach ($c as $k => $v) echo "$k => $v\n";
echo json_encode($c) . PHP_EOL;
echo $c . PHP_EOL;
var_dump(isset($c['b']), $c instanceof Traversable);

// EXERCISE: Collection e map(callable) method jog korun.
```

24. Class Functions

24-class-functions.php

```
<?php
class Animal { public $name = 'Generic'; protected $age = 2; public function speak() {} }
class Dog extends Animal { public $breed = 'Local'; public function fetch() {} }

$d = new Dog();
var_dump(class_exists('Dog'));                  // true
var_dump(method_exists($d, 'fetch'));           // true
var_dump(property_exists('Dog', 'age'));        // true (protected-o dhore)
var_dump(is_a($d, 'Animal'));                   // true
var_dump(is_subclass_of($d, 'Animal'));         // true
echo get_class($d) . PHP_EOL;                   // Dog
echo get_parent_class($d) . PHP_EOL;            // Animal
print_r(get_object_vars($d));                   // public property (scope onujayi)
print_r(get_class_methods('Dog'));              // method list
print_r(class_parents($d));                     // SPL
echo $d::class . PHP_EOL;                       // Dog

// Real use: dynamic method call
$method = 'speak';
if (method_exists($d, $method)) $d->$method();

// EXERCISE: ekta function banan jeta object nie tar shob public method er nam print kore.
```

25. Serialization

25-serialization.php

```
<?php
class Session {
    public function __construct(
        public string $user,
        private string $token,
        public ?PDO $db = null,       // resource-like, serialize kora jay na
    ) {}

    // PHP 7.4+ preferred
    public function __serialize(): array {
        return ['user' => $this->user, 'token' => $this->token];   // db bad
    }
    public function __unserialize(array $d): void {
        $this->user  = $d['user'];
        $this->token = $d['token'];
        $this->db    = null;          // reconnect korte hobe
    }
}

$s = new Session('asif', 'abc123');
$str = serialize($s);
echo $str . PHP_EOL;

$obj = unserialize($str, ['allowed_classes' => [Session::class]]);
echo $obj->user . PHP_EOL;

// Purono style: __sleep() (kon property save hobe), __wakeup() (load er por)

// !!! SECURITY: user input kokhono unserialize() korben na (Object Injection).
// Dorkar hole allowed_classes => false dien, ba json_encode/json_decode use korun.
$safe = unserialize($str, ['allowed_classes' => false]);   // __PHP_Incomplete_Class

// EXERCISE: ekta Cart object serialize kore file e save korun, abar load korun.
```

26. Exceptions

26-exceptions.php

```
<?php
// Custom exception
class InsufficientBalance extends Exception {
    public function __construct(public readonly float $needed, ?Throwable $prev = null) {
        parent::__construct("Need $needed more", 100, $prev);
    }
}

class Account {
    public function __construct(private float $balance) {}
    public function withdraw(float $amt): float {
        if ($amt <= 0) throw new InvalidArgumentException('Amount must be positive');
        if ($amt > $this->balance) throw new InsufficientBalance($amt - $this->balance);
        return $this->balance -= $amt;
    }
}

$acc = new Account(500);
foreach ([200, 1000, -5] as $amt) {
    try {
        echo "Left: " . $acc->withdraw($amt) . PHP_EOL;
    } catch (InsufficientBalance $e) {
        echo "Failed: {$e->getMessage()} (code {$e->getCode()})\n";
    } catch (InvalidArgumentException | TypeError $e) {   // multi-catch
        echo "Invalid: " . $e->getMessage() . PHP_EOL;
    } finally {
        echo "-- done --\n";                              // shob shomoy chole
    }
}

// Error vs Exception: duto-i Throwable implement kore
try { intdiv(1, 0); } catch (DivisionByZeroError $e) { echo "Error: " . $e->getMessage() . PHP_EOL; }

// Exception chaining
try {
    try { throw new RuntimeException('DB down'); }
    catch (RuntimeException $e) { throw new LogicException('Service failed', 0, $e); }
} catch (Exception $e) {
    echo $e->getMessage() . ' <- ' . $e->getPrevious()->getMessage() . PHP_EOL;
}

// Global handler
set_exception_handler(fn(Throwable $t) => print("Uncaught: " . $t->getMessage() . PHP_EOL));

// EXERCISE: ValidationException banan jeta errors array hold kore.
```